

Laboratorio 03 · Memoria, WiredTiger y sistema operativo

Duración: 20 minutos · Carpeta de trabajo: `dia1/`

Objetivos

Vas a observar cómo usa la memoria `mongod`, a leer las métricas principales de la caché de WiredTiger, a comparar algoritmos de compresión y a revisar los avisos de arranque relacionados con el sistema operativo.

Paso 1 · Cargar datos en el standalone

```
docker compose exec toolbox mongosh "mongodb://solo:27017" /scripts-comunes/datos.js
```

Paso 2 · La caché de WiredTiger

```
docker compose exec toolbox mongosh "mongodb://solo:27017"
```

```
> const c = db.serverStatus().wiredTiger.cache
> c["maximum bytes configured"] / 1024 / 1024
> c["bytes currently in the cache"] / 1024 / 1024
> c["tracked dirty bytes in the cache"] / 1024 / 1024
> c["pages read into cache"]
> c["pages evicted by application threads"]
```

Ahora fuerza la lectura de toda la colección y vuelve a mirar los contadores:

```
> db.getSiblingDB("tienda").pedidos.find({ total: { $gt: 99999 } }).itcount()
> db.serverStatus().wiredTiger.cache["bytes currently in the cache"] / 1024 / 1024
```

Y la memoria del proceso en su conjunto:

```
> db.serverStatus().mem
```

1. ¿Cuánto ocupa la caché respecto al máximo configurado?
2. ¿Qué diferencia hay entre `mem.resident` y el tamaño de la caché de WiredTiger?

▼ Ver solución

`mem.resident` es la memoria física total del proceso `mongod` (en MB): incluye la caché de WiredTiger, pero también las conexiones (aproximadamente 1 MB por conexión), las estructuras internas, las agregaciones y ordenaciones en memoria, y la fragmentación del asignador de memoria. Por eso MongoDB reserva por defecto solo el 50 % de (RAM – 1 GB) para la caché: el resto lo necesitan el propio proceso y, sobre todo, la **caché de ficheros del sistema operativo**, donde viven los bloques comprimidos de disco.

Paso 3 · Compresión: snappy frente a zstd

Por defecto WiredTiger comprime las colecciones con **snappy**. Crea una colección con **zstd** y copia en ella los mismos datos:

```
> use tienda
> db.createCollection("pedidos_zstd", { storageEngine: { wiredTiger: {
  configString: "block_compressor=zstd" } } })
> db.pedidos.aggregate([ { $match: {} }, { $merge: { into:
  "pedidos_zstd" } } ])
```

Espera un minuto (o fuerza un checkpoint con `db.adminCommand({ fsync: 1 })`) y compara:

```
> ["pedidos", "pedidos_zstd"].forEach(c => { const s =
  db.getCollection(c).stats({ scale: 1048576 }); print(c, "datos:",
  s.size, "MB  disco:", s.storageSize, "MB") })
```

▼ Ver solución

`size` es el tamaño lógico (sin comprimir) y `storageSize` el ocupado en disco. Con `zstd` normalmente verás un `storageSize` bastante menor que con `snappy`, a cambio de algo más de CPU al comprimir y descomprimir. La compresión se puede elegir por colección (como aquí) o para toda la instancia con `storage.wiredTiger.collectionConfig.blockCompressor`. Usamos `$merge` y no `$out` porque `$merge` inserta en la colección que ya hemos creado con `zstd`, respetando su configuración de almacenamiento.

Paso 4 · Avisos de arranque y ajustes del sistema operativo

```
> db.adminCommand({ getLog: "startupWarnings" }).log.map(l =>
  JSON.parse(l).msg)
```

Desde tu equipo, mira qué ve el contenedor del sistema operativo (en Docker Desktop es el kernel de la máquina virtual de Docker):

```
docker compose exec solo bash -c "cat
/sys/kernel/mm/transparent_hugepage/enabled; ulimit -n; ulimit -u; cat
/proc/sys/vm/swappiness; df -hT /data/db"
docker stats --no-stream solo
```

Rellena esta tabla con lo que observes y con lo que recomendarías en un servidor de producción:

Ajuste	Valor en el laboratorio	Recomendación en producción
Transparent Huge Pages		
Límite de ficheros abiertos (<code>nofile</code>)		
Límite de procesos (<code>nproc</code>)		
<code>vm.swappiness</code>		
Sistema de ficheros de <code>/data/db</code>		

▼ Ver solución

Para MongoDB 8.0 en Linux la recomendación sobre THP cambió: ahora se recomienda **habilitarlo** (`always`), porque la nueva versión de TCMalloc lo aprovecha. En 7.0 y anteriores la recomendación era deshabilitarlo. Los límites `nofile` y `nproc` deberían ser al menos 64000. `vm.swappiness` entre 1 y 10 para que el kernel no mande a swap memoria de `mongod`. El sistema de ficheros recomendado con WiredTiger es **XFS**, montado con `noatime`. En Docker verás `overlay` para el sistema de ficheros del contenedor y `ext4` u otro para el volumen; en producción, los datos deben ir en un disco dedicado. Si ves avisos sobre `vm.max_map_count`, `NUMA` o `glibc.pthread.rseq`, son ajustes del host que se aplican en el servidor, no dentro del contenedor.

Para pensar

Un servidor de 64 GB de RAM ejecuta solo `mongod`. ¿Qué tamaño de caché de WiredTiger tendrá por defecto? ¿Y si en ese servidor ejecutas tres instancias de `mongod` sin configurar nada?

▼ Ver solución

Por defecto: 50 % de $(64 - 1)$ GB = 31,5 GB. Con tres instancias sin configurar, cada una intentaría quedarse con 31,5 GB y el servidor acabaría haciendo swap o siendo víctima del OOM killer. Cuando hay varias instancias por máquina, o en contenedores, hay que fijar `cacheSizeGB` explícitamente.